RNN#

https://pytorch.org/docs/stable/generated/torch.nn.modules.rnn.RNN.html

Description:

Apply a multi-layer Elman RNN with tanh\tanhtanh or ReLU\text{ReLU}ReLU non-linearity to an input sequence. For each element in the input sequence, each layer computes the following function: where hth_tht is the hidden state at time t, xtx_txt is the input at time t, and h(t−1)h_{(t-1)}h(t−1) is the hidden state of the previous layer at time t-1 or the initial hidden state at time 0. If nonlinearity is 'relu', then ReLU\text{ReLU}ReLU is used instead of tanh\tanhtanh. input_size – The number of expected features in the input x hidden_size – The number of features in the hidden state h num_layers – Number of recurrent layers. E.g., setting num_layers=2 would mean stacking two RNNs together to form a stacked RNN, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1

Function Signature

```
class torch.nn.modules.rnn.RNN(input_size, hidden_size,
num_layers=1, nonlinearity='tanh', bias=True,
batch_first=False, dropout=0.0, bidirectional=False,
device=None, dtype=None)[source]#

Parameters

Inputs: input, hx

Outputs: output, h_n

Variables
```

Code Examples

```
# Efficient implementation equivalent to the following with
bidirectional=False
rnn = nn.RNN(input_size, hidden_size, num_layers)
params = dict(rnn.named_parameters())
def forward(x, hx=None, batch_first=False):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(rnn.num_layers, batch_size,
rnn.hidden_size)
    h_t_minus_1 = hx.clone()
    h_t = hx.clone()
    output = []
    for t in range(seq_len):
        for layer in range(rnn.num_layers):
            input_t = x[t] if layer == 0 else h_t[layer - 1]
            h_t[layer] = torch.tanh(
                input_t @ params[f"weight_ih_l{layer}"].T
                + h_t_minus_1[layer] @
params[f"weight_hh_l{layer}"].T
                + params[f"bias_hh_l{layer}"]
                + params[f"bias_ih_l{layer}"]
            )
        output.append(h_t[-1].clone())
        h_t_minus_1 = h_t.clone()
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

```
>>> rnn = nn.RNN(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> h0 = torch.randn(2, 3, 20)
>>> output, hn = rnn(input, h0)
```

Notes & Warnings

NOTE Note All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

NOTE Note For bidirectional RNNs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when batch_first=False: output.view(seq_len, batch, num_directions, hidden_size).

NOTE Note batch_first argument is ignored for unbatched inputs.

⚠ WARNING Warning There are known non-determinism issues for RNN functions on some versions of cuDNN and CUDA. You can enforce deterministic behavior by setting the following environment variables: On CUDA 10.1, set environment variable CUDA_LAUNCH_BLOCKING=1. This may affect performance. On CUDA 10.2 or later, set environment variable (note the leading colon symbol) CUBLAS_WORKSPACE_CONFIG=:16:8 or CUBLAS_WORKSPACE_CONFIG=:4096:2. See the cuDNN 8 Release Notes for more information.

NOTE Note If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype torch.float16 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.

Detailed Documentation

Documentation

Apply a multi-layer Elman RNN with tanh\tanhtanh or ReLU\text{ReLU}ReLU non-linearity to an input sequence. For each element in the input sequence, each layer computes the following function:

where hth_tht is the hidden state at time t, xtx_txt is the input at time t, and h(t−1)h_{(t-1)}h(t−1) is the hidden state of the previous layer at time t-1 or the initial hidden state at time 0. If nonlinearity is 'relu', then ReLU\text{ReLU}ReLU is used instead of tanh\tanhtanh.

input_size – The number of expected features in the input x

hidden_size – The number of features in the hidden state h

num_layers – Number of recurrent layers. E.g., setting num_layers=2 would mean stacking two RNNs together to form a stacked RNN, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1

nonlinearity – The non-linearity to use. Can be either 'tanh' or 'relu'. Default: 'tanh'

bias – If False, then the layer does not use bias weights b_ih and b_hh. Default: True

batch_first – If True, then the input and output tensors are provided as (batch, seq, feature) instead of (seq, batch, feature). Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: False

dropout – If non-zero, introduces a Dropout layer on the outputs of each RNN layer except the last layer, with dropout probability equal to dropout. Default: 0

bidirectional – If True, becomes a bidirectional RNN. Default: False

input: tensor of shape (L,Hin)(L, H_{in})(L,Hin) for unbatched input, (L,N,Hin)(L, N, H_{in}) (L,N,Hin) when batch_first=False or (N,L,Hin)(N, L, H_{in})(N,L,Hin) when batch_first=True containing the features of the input sequence. The input can also be a packed variable length sequence. See torch.nn.utils.rnn.pack_padded_sequence() or torch.nn.utils.rnn.pack_sequence() for details.

hx: tensor of shape (D∗num_layers,Hout)(D * \text{num_layers}, H_{out}) (D∗num_layers,Hout) for unbatched input or (D∗num_layers,N,Hout)(D * \text{num_layers}, N, H_{out})(D∗num_layers,N,Hout) containing the initial hidden state for the input sequence batch. Defaults to zeros if not provided.

output: tensor of shape (L,D∗Hout)(L, D * H_{out})(L,D∗Hout) for unbatched input, (L,N,D∗Hout)(L, N, D * H_{out})(L,N,D∗Hout) when batch_first=False or (N,L,D∗Hout)(N, L, D * H_{out})(N,L,D∗Hout) when batch_first=True containing the output features (h_t) from the last layer of the RNN, for each t. If a torch.nn.utils.rnn.PackedSequence has been given as the input, the output will also be a packed sequence.

h_n: tensor of shape (D∗num_layers,Hout)(D * \text{num_layers}, H_{out}) (D∗num_layers,Hout) for unbatched input or (D∗num_layers,N,Hout)(D * \text{num_layers}, N, H_{out})(D∗num_layers,N,Hout) containing the final hidden state for each element in the batch.

weight_ih_l[k] – the learnable input-hidden weights of the k-th layer, of shape (hidden_size, input_size) for k = 0. Otherwise, the shape is (hidden_size, num_directions * hidden_size)

weight_hh_l[k] – the learnable hidden-hidden weights of the k-th layer, of shape (hidden_size, hidden_size)

bias_ih_l[k] – the learnable input-hidden bias of the k-th layer, of shape (hidden_size)

bias_hh_l[k] – the learnable hidden-hidden bias of the k-th layer, of shape (hidden_size)

All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k ,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

For bidirectional RNNs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when batch_first=False: output.view(seq_len, batch, num_directions, hidden_size).

batch_first argument is ignored for unbatched inputs.

There are known non-determinism issues for RNN functions on some versions of

cuDNN and CUDA. You can enforce deterministic behavior by setting the following environment variables:

On CUDA 10.1, set environment variable CUDA_LAUNCH_BLOCKING=1. This may affect performance.

On CUDA 10.2 or later, set environment variable (note the leading colon symbol) CUBLAS_WORKSPACE_CONFIG=:16:8 or CUBLAS_WORKSPACE_CONFIG=:4096:2.

See the cuDNN 8 Release Notes for more information.

If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype torch.float16 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.

Runs the forward pass.
